COMP 382: Reasoning about Algorithms
Homework 1

Konstantinos Mamouras Michael Burke

Due on September 11, 2022

(released on September 1, 2022)

1 Radix Trees [20 points]

This problem is based on Problem 12.2 from CLRS. A *radix tree* is a data structure for the efficient representation of sets of strings. The following radix tree data structure stores the bit strings 1011, 10, 011, 100, and 0.

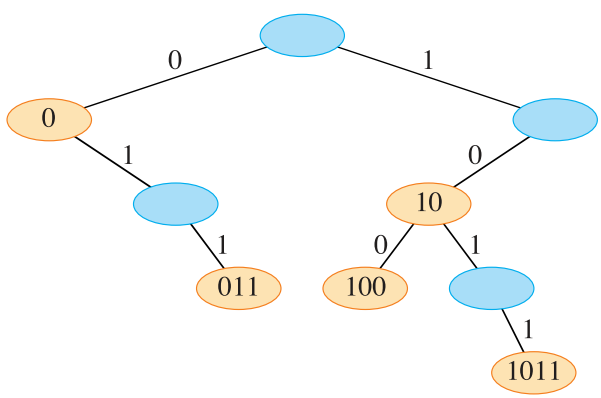

Figure 12.5 A radix tree storing the bit strings 1011, 10, 011, 100, and 0. To determine each node's key, traverse the simple path from the root to that node. There is no need, therefore, to store the keys in the nodes. The keys appear here for illustrative purposes only. Keys corresponding to blue nodes are not in the tree. Such nodes are present only to establish a path to other nodes.

(1) [**3 points**] Define formally the radix tree data structure.

(2) [**10 points**] Let T be a radix tree and w be a bit string. Write the pseudo-code of RADIX-TREE-SEARCH(T, w) and RADIX-TREE-INSERT(T, w) so that each of the functions takes $\Theta(|w|)$ time. Justify that your algorithms indeed take $\Theta(|w|)$ time.

(3) [**7 points**] Given two strings $a = a_0 a_1 \ldots a_p$ and $b = b_0 b_1 \ldots b_q$, where each a_i and b_j is in some ordered alphabet, we say that string a is lexicographically less than string b if either

(i) there exists an integer j, where $0 \leq j \leq \min\{p, q\}$, such that $a_i = b_i$ for all $i = 0, 1, \ldots, j-1$ and $a_j < b_j$, or

(ii) $p < q$ and $a_i = b_i$ for all $i = 0, 1, \ldots, p$.

Let S be a set of distinct bit strings whose lengths sum to n (i.e., the sum of the lengths of all strings in S equals n). Show how to use a radix tree to sort S lexicographically in $\Theta(n)$ time. Describe the algorithm idea in English, but please make sure your description is very clear. Justify that your algorithm indeed takes $\Theta(n)$ time.

2 Multisets [20 Points]

Let K be a set of elements. A *(finite) multiset* or *bag* over K is a collection of elements from K, each of which can occur any finite number of times. The *count* of an element $a \in K$ in a multiset S is the number of occurrences of a in S. The count of an element that does not appear in the multiset is equal to 0. For example, the multiset $S = \{a, b, b, b, c, c, d\}$ contains the elements a, b, c, d with counts $1, 3, 2, 1$ respectively. The count of e in S is 0.

Part 1 [10 points]: Design a data structure that can represent finite multisets and can support the following operations with the specified time complexities.

Operation	Description	Time (worst-case)
$\textsc{Create}(S)$	Set S to be the empty multiset	$O(1)$
$\textsc{Insert}(S, k)$	Insert the element k in the multiset S	$O(\log n)$
$\textsc{GetCount}(S, k)$	Return the count of k in S	$O(\log n)$
$\textsc{Delete}(S, k)$	Delete one occurrence of k from S	$O(\log n)$
$\textsc{GetCounts}^{\geq}(S, k)$	Return the total count of elements $\geq k$ in S	$O(\log n)$

Assume that the elements are linearly ordered. In the table above, n is the number of unique elements in the multiset.

Part 2 [10 points]: Adapt the multiset data structure to support all operations from Part 1, as well as the following ones:

Operation	Description	Time (worst-case)
$\textsc{WithCount}(S, c)$	Return the number of elements with count c in S	$O(\log n)$
$\textsc{Frequent}(S, c)$	Return the number of elements with count $\geq c$ in S	$O(\log n)$

In the table above, n is the number of unique elements in the multiset.

Notes for both parts: Try to make your data structure as efficient as possible in terms of information stored. Describe the design of the data structure, and briefly describe in English how you would implement each of the operations so that it has the prescribed worst-case time complexity.

You can assume a known fact for AVL trees (in case you use AVL trees for your implementation). There is an algorithm for performing node deletion in an AVL tree with running time $O(\log n)$, where n is the size of the tree.

3 AVL Trees: Height & Size [10 Points]

In class, we discussed the main ideas that are needed to prove that an AVL tree with n nodes has height $O(\log n)$. Give the best upper bound you can find on the height of an AVL tree with n nodes. The upper bound that you give should be of the form $c \cdot \log_2(n+k)+b$, where c, k, b are constants. Provide a careful and very thorough justification of every step.

Note: Recall that for AVL trees we use the following definition: The *height* of a tree is the number of nodes in the longest path from the root to a leaf.